

访谈笔记

访谈笔记：GUI Agent 的下一站

基于 Ungrounded 不着边际 EP01 整理

2026-05-08



视频作者/频道：Ungrounded 不着边际

发布日期：2026-01-28

视频时长：02:27:24

视频链接：<https://www.bilibili.com/video/BV1y36jBnEiD/>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 从“会操作电脑”到“理解数字任务”	2
1.1 本章小结	3
2 Benchmark 危机：从可比较分数到真实场景信号	4
2.1 本章小结	5
3 从广告入口到智力劳动：Agent 的商业模式重估	5
3.1 本章小结	6
4 产品形态：从 Operator 到 UDA	6
4.1 本章小结	7
5 长任务的悖论：Agent 越能跑，用户越容易累	7
5.1 本章小结	8
6 持续学习与记忆：上线后的模型不该死掉	8
6.1 本章小结	9
7 知识、符号与组织语义库	9
7.1 本章小结	10
8 Startup 护城河：速度之后留下什么	10
8.1 本章小结	11
9 总结：GUI Agent 的下一站不是更会点屏幕	11

导言

这场对话的价值不在于给出一个“GUI Agent 现状速览”，而在于它把同一个问题从研究、产品、商业和组织四个层面反复拆开：当模型终于开始能够操作真实数字环境时，我们究竟应该把它理解成一个会点屏幕的自动化工具，还是理解成一种正在形成的新型数字代理？谢天宝、谷雨和尚晏仪的背景正好构成三种互补视角：一个从 OSWorld、GUI grounding 和 benchmark 出发，一个从 semantic parsing、Mind2Web 和学术 evaluation 出发，一个从 2021 年开始做 GUI Agent 产品、后来转向 AI 旅行的商业实践出发。三个人并没有给出一个统一口号，但他们不断围绕同一组张力来回校准：真实场景与 benchmark、通用入口与垂直产品、可见操作与用户体验、长期记忆与概念漂移、速度优势与可积累资产。

如果只按时间顺序复述这期节目，会很容易把它读成一串行业观点：Operator 不好用、benchmark 分数没那么重要、Agent 可能冲击广告、memory 很难、startup 要有护城河。但这场对话真正有意思的地方，是这些观点并不是孤立的。Operator 的体验问题，最后会回到“用户到底想不想看 Agent 操作”；benchmark 的危机，最后会回到“真实产品能否产生可靠 reward”；Unified Action Space 的讨论，最后会回到“Deep Research、Coding、Browser Use、GUI 操作为什么不该被切碎”；概念漂移的警惕，最后会回到“一个组织如何维持共同语义”。换句话说，这不是一场关于 GUI Agent 的功能清单，而是一场关于“数字代理如何进入现实世界”的方法论讨论。

这份笔记因此不严格按时间线展开，而是按八个主题重新组织。每个主题都尽量保留对话里的具体语境：谁从什么经验出发，为什么会得出这个判断，它反驳了哪种常见误解，以及这个判断对研究者、产品团队或创业者分别意味着什么。

阅读这份笔记的三条主线

第一条主线是“评价问题”：从 OSWorld、Mind2Web 到真实产品，什么样的 evaluation 才能真正指导 Agent 进步。第二条主线是“产品问题”：从 Operator、豆包手机、Unified Action Space 到 context switching，用户到底需要一个怎样的数字代理。第三条主线是“积累问题”：从 memory、knowledge、组织语义库到 startup 护城河，Agent 系统如何在真实交互中形成长期资产。

最后要说明的是，原视频没有官方字幕，本整理基于本地 Whisper 转写和逐段校读完成。由于视频主要是多人对话而非课件讲授，正文没有强行插入人像截图；封面保留原始视频封面，图表主要用于压缩概念关系，而不是装饰。

1 从“会操作电脑”到“理解数字任务”

三位嘉宾的进入路径，恰好说明 GUI Agent 并不是一个突然冒出来的新风口。谷雨从 semantic parsing 讲起：早期研究关注如何把自然语言 instruction 转换成数据库查询、知识库操作或其他 formal representation。那个阶段的环境更可控，任务更像“把一句话翻译成一个可执行结构”。但问题本质已经很接近今天的 computer-using agent：用户给出自然语言目标，系统需要把它转成环境中的可执行动作。

谢天宝的路径则沿着 agent benchmark 与真实 GUI 环境展开。从 AgentBench、intercode、OSWorld 到多模态 GUI grounding，他关注的是怎样让模型在屏幕、网页、操作系统里真正做事，而不是只在文本空间里回答问题。尚晏仪提供了第三条线：从 2021 年开始做能替用户操作电脑的产品，到后来做 Web Agent 和通用 GUI Agent，再到发现通用入口太难商业化，转向 AI 旅行这类具体场

景。三条线放在一起看，会发现 GUI Agent 的关键不是“鼠标点击”本身，而是语言、视觉、动作、用户偏好和真实业务流程如何被接到一起。

GUI Agent 的历史前传

前传线索	对今天 GUI Agent 的意义
Semantic Parsing	训练系统把自然语言任务转成可执行表示，提供了“语言到行动”的早期形式。
Web Automation / RPA	证明很多数字任务可以被流程化，但也暴露了规则系统对动态环境的脆弱性。
Agent Benchmark	用 OSWorld、Mind2Web 等任务把能力问题显式化，让模型可以被横向比较。
产品化探索	暴露研究 demo 与用户体验之间的距离：能执行不等于好用，能展示过程不等于建立信任。

2025 年的变化，是这些线索终于被更强的多模态模型重新接上。谢天宝提到，早期模型连“点击右上角关闭按钮”“把音量拖到 80%”“修改输入框里的名字”这类看似简单的动作都做不好。后来通过大规模合成数据、环境扩展和模型迭代，GUI grounding 从“纯概念”走向“基本可用”。这不是小修小补，因为 GUI 环境的难点并不只是看见按钮，而是把视觉定位、语言指令、屏幕坐标、动作类型和任务目标同时对齐。

但这也正是产品问题开始变尖锐的地方。尚晏仪讲 Operator 时的态度很典型：它让从业者感到方向被认可，但她并不认为它是一个好产品。用户看着一个小电脑慢慢点击网页，很容易迷茫；一个原本自己几秒钟能完成的动作，被 AI 以很慢的速度展示出来，未必增加信任，反而可能暴露笨拙。研究者想证明“我真的替你操作了”，用户却可能只想知道“结果是否可靠，我是否需要确认”。

把 GUI Agent 等同于“看见屏幕并点击”，会错过真正的问题

GUI Agent 的表层动作是点击、输入、滚动和拖拽；但它的深层问题是数字任务理解。一个成熟 Agent 应该知道什么时候点 GUI，什么时候调用 API，什么时候写脚本，什么时候直接读文件，什么时候向用户确认。GUI 是动作空间的一部分，而不是智能本身。

这一章真正建立的是整场对谈的底层前提：GUI Agent 的未来不只是更强 grounding，也不是更逼真的“远程小电脑”。它要解决的是数字环境里的目标理解、行动选择、反馈学习和信任边界。只有把这个前提立住，后面关于 benchmark、商业模式、UDA、memory 和护城河的讨论才会连成一条线。

1.1 本章小结

GUI Agent 不是从零出现的新概念，而是 semantic parsing、web automation、agent benchmark 和产品化探索汇合后的结果。2025 年多模态能力让“操作电脑”进入可用区间，但真正难的是从“执行动作”上升到“理解数字任务”。Operator 这类产品的争议说明，研究证明和用户体验之间仍有很长距离。

2 Benchmark 危机：从可比较分数到真实场景信号

这场对话最扎实的一段，是关于 benchmark 和 evaluation。谷雨的起手式很清楚：学术界和工业界需要的 evaluation 本来就不是同一种东西。学术界希望有可控实验来验证假设、定义新问题、比较方法能力；工业界则更关心评测上的提升能否迁移到真实部署，是否真的让用户完成任务、满意、留存或付费。

这一区分重要，是因为当下很多人一边说 benchmark 分数不重要，一边又离不开 benchmark。谢天宝给了一个更细的判断：benchmark 不是没用，而是要看处在什么阶段。在 0 到 1 阶段，它能定义问题、暴露短板、证明现有模型不会；一旦进入 60 到 99 的优化阶段，公开分数就越来越容易受数据合成、训练集过拟合和任务定制影响。这时只看分数，反而会误判真实可用性。

Benchmark 的位置变化

Benchmark 最适合扮演 signal，而不是 final answer。它可以告诉社区“这里有一个问题”“模型在这里有短板”“某种能力开始出现”；但它不能单独回答“这个 Agent 在真实产品里是否好用”。越接近产业落地，越需要真实用户、真实 API、真实 case 和持续反馈闭环来校准。

尚晏仪从旅行产品讲 reward，补上了工业界最难的一环。旅行产品最终可以看“有没有卖出一张票”，但这个 reward 离模型的每一步决策太远。用户没买票，可能是价格不合适，可能是路线不喜欢，可能是不信任 AI，也可能只是临时改变计划。用户说“不错”，也不等于真的会购买。也就是说，最终结果很清楚，但过程归因很模糊；主观反馈很多，但可训练信号很少。

这和学术 benchmark 的困难不同。学术任务里 reward 往往是预先定义的：做对得分，做错扣分。但真实产品里，用户自己也未必知道什么体验才是好体验。例如语音旅行助手什么时候该打断，什么时候该倾听，什么时候该长篇解释，这些问题不可能靠研究者凭直觉写一个 benchmark 就解决。尚晏仪说他们更依赖 A/B Test 和可持续试验基础设施，其实是在承认：用户偏好不是写出来的，是在真实交付中逐渐显影的。

0 到 1 与 60 到 99

阶段	Benchmark 的价值	主要风险
0 到 1	定义新任务，证明模型不会，给领域建立共同语言。	任务可能脱离真实场景，成为“为了 benchmark 而 benchmark”。
60 到 99	作为局部能力信号，帮助工程团队定位短板。	分数容易被训练、数据增强、任务定制和选择性展示污染。
真实产品	需要和用户反馈、商业结果、失败归因一起使用。	reward 稀疏、主观、长链路，难以直接转成训练信号。

这一段背后的更大问题，是 Agent 研究开始从“能力是否存在”走向“能力如何稳定产生价值”。在前者里，benchmark 是一种科学仪器；在后者里，benchmark 只是产品运营系统的一部分。真正难的不是做出一个分数，而是把分数、用户反馈、人工分析、A/B Test、模型选择、prompt 策略、工具流程和业务指标连成循环。

不要把产品失败归因给一个单一模型分数

Agent 产品失败时，很可能不是“模型不够强”这么简单。也许是任务定义错了，也许是 reward 太远，也许是交互不可信，也许是用户根本不想以这种方式完成任务。Benchmark 只能照亮一部分原因，不能替代产品归因。

2.1 本章小结

Benchmark 危机的本质不是分数失去意义，而是分数的解释边界变窄了。学术界仍需要 benchmark 定义问题，工业界则必须把 evaluation 放进真实产品闭环里。GUI Agent 的评价尤其困难，因为它面对长链路任务、主观体验和稀疏 reward；真正有价值的 infrastructure，是能不断试、不断归因、不断把用户反馈转化为系统改进的 infrastructure。

3 从广告入口到智力劳动：Agent 的商业模式重估

商业模式讨论从一个很经典的对照开始：互联网时代的大模式主要是广告、订阅和交易抽佣。广告卖的是注意力，订阅卖的是工具效率，抽佣卖的是交易撮合。Agent 作为入口时，会直接挑战广告，因为广告系统希望用户停留、浏览、点击，而一个好 Agent 恰恰应该减少用户停留时间，替用户完成筛选、比较和执行。

尚晏仪的判断很有启发性：互联网完成的是信息连接，广告费对应的是信息分发；Agent 输出的是智力劳动，而人类社会里智力劳动通常通过工资、服务费、中介费、佣金等形式付费。这就是为什么她更看好订阅和抽佣，而不是把 Agent 强行塞回广告逻辑。Agent 如果真的像秘书、顾问或代理，它的商业价值就应该来自节省时间、降低决策成本、撮合交易和承担行动结果，而不是让用户多看几秒内容。

旅行场景之所以在她的实践里成立，也正因为它天然存在 travel agent 这个历史角色。用户理解“代理”意味着什么，也能接受为规划、筛选、交易和服务付费。旅行还拥有明确交易闭环：酒店、机票、路线、签证、行程服务都能连接到抽佣或服务费。这和泛化的“超级入口”不同，后者看起来空间大，但 reward、履约和商业闭环都更模糊。

Agent 更像劳动力，而不是广告位

如果一个系统的价值是替用户思考、筛选、行动和承担流程成本，那么它更接近智力劳动。广告逻辑要求占用用户注意力，Agent 逻辑要求释放用户注意力。两者并非不能结合，但默认商业重心不同。

豆包手机的讨论，把这个问题推到更高一层。手机级 Agent 的优势非常清楚：手机掌握用户住址、偏好、账号、聊天、日程和生活习惯，因此最有机会成为生活秘书。但它同时会触碰现有 App 分发、广告推荐和交易入口的利益结构。换句话说，手机 Agent 技术上很合理，商业上却会改变很多人的饭碗。

这里有一个细节很值得抓住。尚晏仪说，如果用户直接告诉 Agent “我想去海岛”，这比短视频广告系统观察到用户在海岛视频上多停留几秒更强。因为前者是明确意图，后者只是行为推断。但明确意图不自动等于商业成功。Agent 还要把意图结构化，找到供应，比较选项，建立信任，并完成交易。广告系统擅长把模糊行为变成推荐，Agent 系统则必须把明确语言变成可靠行动。

从注意力经济到代理经济

模式	价值来源	与 Agent 的关系
广告	用户停留、浏览、点击、曝光。	好 Agent 会减少停留，因此天然冲突。
订阅	用户为效率、工具能力和可靠性付费。	适合高频、明确、可持续提效场景。
抽佣	平台撮合交易并参与履约。	适合旅行、采购、电商、招聘等有交易闭环的场景。
代理服务	用户为智力劳动、筛选、执行和责任边界付费。	最接近强 Agent 的长期形态。

3.1 本章小结

Agent 产品不能简单复刻互联网广告模式。它的价值更像智力劳动、代理服务和交易撮合。旅行之所以适合作为案例，是因为它既有传统代理角色，也有清晰交易闭环。手机 Agent 的潜力来自个人上下文，但真正难的是把上下文转成可信行动，同时处理与现有生态的商业冲突。

4 产品形态：从 Operator 到 UDA

如果说商业模式回答“Agent 怎么赚钱”，产品形态回答的就是“用户到底想用什么”。在这一点上，三位嘉宾并没有把 GUI Agent 想象成一个永远在屏幕上可见的机器人。谷雨想要的是生活秘书，能处理签证、日程、出行、杂事；尚晏仪想要的更像人生教练或一面镜子，能结合长期 context 帮她整理信息、做事和反馈。共同点是：他们都不执着于看 Agent 逐步点击，而是希望 Agent 围绕目标组织行动。

这正是 UDA 和 Unified Action Space 出现的语境。Deep Research、Coding、Browser Use、GUI Agent 被做成不同产品，是今天行业分工的结果，但用户的真实目标并不按这些产品边界切开。研究一个问题可能需要搜网页、读 PDF、写代码、整理文件、发消息；订一次旅行可能需要读聊天记录、查签证、比较酒店、调用地图、打开 OTA。用户的目标发生在同一个数字环境里，产品却要求用户在多个 Agent 之间搬运上下文，这显然不是终局。

Unified Action Space 的真正含义

Unified Action Space 不是“把所有按钮放进一个超级工具栏”，而是让 Agent 根据目标选择行动方式。能调 API 就不要模拟点击，能写脚本就不要逐页操作，能直接读文件就不要截图 OCR，必须进入 GUI 时再进入 GUI。统一的不是界面，而是围绕用户目标组织动作的能力。

这一点也解释了为什么“Computer-Using Agent”比“GUI Agent”更准确。GUI use 是 computer use 的子集。把 Agent 的能力绑定在视觉点击上，会让系统在很多场景里故意变笨。真正成熟的数字代理应该能在 API、代码、文件、浏览器、GUI、消息和用户确认之间切换，而不是把所有任务都翻译成鼠标操作。

生成式 UI 的讨论，则补上了另一半。尚晏仪并不完全看好“所有界面都由模型即时生成”，因为用户需要稳定性和可预测性。界面不只是信息容器，也是用户习惯和信任的结构。如果每次按钮位置、交互方式和结果展示都不一样，用户会不安，也很难形成熟练操作。更现实的方向可能是稳

定组件加动态编排：基础操作、确认流程、权限边界保持稳定，具体结果呈现和任务辅助可以动态生成。

生成式 UI 的误区

生成式 UI 的价值不是让每次界面都变得陌生，而是让界面更贴近当前任务。若牺牲可预测性来追求“AI 感”，产品反而会变难用。对 Agent 来说，稳定的语义组件比炫目的即时生成更重要。

这里还有一个被上一版摘要低估的问题：隐私与信任。生活秘书式 Agent 越有用，就越需要访问住址、账号、聊天、日程、偏好、支付和身份信息。它可以本地化，可以联邦学习，可以像密码管理器一样建立信任边界，但无论哪种路线，都必须让用户知道什么被记住、什么可删除、什么动作需要确认。Agent 的产品形态不是 UI 形态这么简单，它本质上是权限形态。

4.1 本章小结

从 Operator 到 UDA，变化的是产品中心：从“展示 AI 如何操作”转向“围绕用户目标组织行动”。Unified Action Space 要解决能力碎片化和上下文搬运问题；Computer-Using Agent 要避免把所有任务降级成 GUI 点击；生成式 UI 要服务任务，而不是破坏可预测性。真正成熟的 Agent 产品，必须同时处理行动选择、用户确认和隐私边界。

5 长任务的悖论：Agent 越能跑，用户越容易累

这一期里最贴近日常体验的观察，是 context switching。尚晏仪说，她大量使用 Deep Research 和 coding agent 后，感到一个很强的负担：一个完整 prompt 丢出去，Agent 会运行很久；如果能力差，她不会用；如果能力好，她又不需要看中间输出。于是她不知道等待时该做什么，只好切到别的任务，几分钟后再回来检查，反而更累。

这个观察非常重要，因为它反驳了很多 Agent 产品的默认叙事。今天不少产品会把“能做一小时任务”当成卖点，但谷雨指出，如果同样质量能 30 秒完成，没人想等一小时。长任务能力当然有价值，但它是系统能力，不是用户体验本身。把慢包装成自主性，是一种很危险的产品叙事。

长任务能力不是长等待体验

用户想要的是任务被可靠完成，而不是被迫管理一个长时间运行的黑盒。Agent 能运行很久，说明它可能具备自主执行能力；但产品价值来自更少等待、更少检查、更少认知切换和更高结果质量。

对谈里还把这个问题的产品层和能力层。产品层可以通过中间状态、进度反馈、可中断机制和关键确认点改善体感。例如有的 coding agent 会持续告诉用户正在做什么，而有的系统几分钟没有输出，用户就会焦虑。但能力层更根本：Agent 找信息慢、工具使用不熟、决策路径绕、容易重复打开无用网页，说明它还不具备熟练人类那种快速判断信息价值的能力。

这也解释了为什么“写一个 prompt 让 Agent 跑六小时”只适合极少数用户。真实任务往往需要来回澄清和迭代。用户自己也未必能一次性写出完整需求。一个 Agent 如果拿着一句模糊需求跑很久，最后很可能产生大量看似努力、实则偏离的工作。真正的自主性不应是“长时间独自运行”，而应是“知道什么时候推进、什么时候停下、什么时候请求用户判断”。

长任务体验的四个设计点

- 状态压缩：告诉用户当前在解决什么问题，而不是倾倒底层日志。
- 检查点：在高风险分叉处请求确认，而不是最后一次性交付惊喜。
- 可中断：允许用户改向、暂停、补充信息。
- 差异摘要：完成后说明做了什么、改变了什么、还有什么不确定。

5.1 本章小结

Agent 产品的一个新矛盾是：能力越像“长时间自主执行”，越可能把用户推入频繁检查和上下文切换。产品层可以通过反馈设计缓解，但根本仍然是模型的 tool use、decision making 和任务判断能力。长期来看，用户不应被训练去适应低效 Agent；Agent 应该学会减少用户的认知负担。

6 持续学习与记忆：上线后的模型不该死掉

对谈后半段转向研究问题，密度明显变高。谢天宝提出，过去一年 GUI Agent 很多工作更像工程补坑：补数据、补 infra、补 evaluation、补环境。如果要寻找更有范式变化的方向，online learning、continual learning、test-time training 和 self-evolving 可能是一两年内的重要问题。

这个判断的背景是，今天的模型训练范式仍然很“离线”：pre-training、SFT、RL，然后上线推理。上线之后，模型基本不变。即使每天有海量用户交互，模型也不会实时从这些交互中获得能力增益。最多是日志回流、离线清洗、再训练、再部署。问题在于真实产品数据噪声极大，用户反馈稀疏而主观，直接变成训练数据并不容易。

Cursor 的 online RL 被拿出来作为一个信号：当产品场景有高频反馈、明确目标和快速部署 infrastructure 时，模型可以更快进入“产品运营即学习”的循环。但 GUI Agent 比 tab prediction 难得多。它面对的是长程任务、开放动作空间、多工具调用、稀疏 reward 和复杂失败归因。

上线后的模型不该只是推理工具

如果模型每天与大量用户交互，却不能从交互中更新知识、偏好、策略或记忆，那么推理过程就只是在消耗算力，而没有产生能力复利。持续学习的目标，是让真实交互成为系统资产，而不是日志垃圾。

Memory 讨论是这一段的核心。谷雨把它拆成三个问题：memory 怎么表示，怎么更新，怎么使用。这个拆法很关键，因为很多产品把 memory 简化成“把历史塞进上下文”或“做一个向量库”，但真正的长期记忆远不止检索。它要决定哪些信息值得长期保留，如何压缩，如何被用户编辑，如何被任务调用，如何避免旧信息污染新决策。

嘉宾们在参数记忆与离散记忆之间来回讨论。参数记忆更新成本高、可塑性差、难以审计；离散记忆如文本、SOP、workflow、knowledge graph 更可编辑、可检查，但也容易信息损耗、结构僵硬、context management 困难。尚晏仪倾向于很多知识要用文本表示，因为文本能进入组织协作；谢天宝则提醒，人脑本身不是纯符号系统，最终可能仍要回到 sub-symbolic 形式。这不是分歧，而是说明未来系统大概率是混合的。

Memory 的三问

问题	关键含义
如何表示	参数、文本、向量、SOP、workflow、knowledge graph 各有不同的可塑性、可审计性和调用成本。
如何更新	从交互中抽取什么，谁来确认，错误记忆如何删除，偏好变化如何覆盖旧记录。
如何使用	是用于检索、规划、个性化、权限控制、错误恢复，还是进入下一轮训练。

Test-time training 和 intrinsic reward 的讨论，则指向另一种可能：模型在推理过程中能否从自身分布、confidence、采样路径或交互结果里获得信号。这里的直觉是，模型在 pre-training 和 post-training 中已经吸收了大量知识，正确答案可能在分布里，只是 decoding 或搜索没有把它取出来。这个方向还远未成熟，但它把“推理”和“学习”的边界重新打开了。

长上下文不是长期记忆

把所有历史都塞进 context 会遇到成本、噪声、隐私、检索和一致性问题。长期记忆必须能压缩、能编辑、能遗忘、能审计；否则它不是资产，而是越来越重的认知债务。

6.1 本章小结

持续学习讨论把 GUI Agent 从一次性工具推向长期系统。上线后的模型若不能从交互中积累能力，就无法形成真正复利。Memory 不是聊天记录，至少包含表示、更新和使用三件事。未来更可能是参数记忆、离散记忆、workflow 与 test-time adaptation 的混合，而不是单一长上下文或单一向量库。

7 知识、符号与组织语义库

知识这一段看似哲学，其实非常工程。谷雨提到，知识至少可以粗略分成事实性知识、过程性知识和情景性知识。事实性知识回答“是什么”，过程性知识回答“怎么做”，情景性知识记录“发生过什么”。对 Agent 来说，最难的往往不是事实，而是流程和情景：如何申请签证、如何在企业系统里报销、如何按某个团队的惯例修代码、如何根据某个用户的偏好安排旅行。

这就引出 workflow 和 neuro-symbolic。让模型每一步都 plan，灵活但不稳定；把流程写成 code、SOP 或 RPA，稳定但死板。真正可用的系统，很可能要在两者之间来回转化：模型从交互中抽象出 documentation、SOP、workflow、knowledge graph，再在执行时用这些结构约束模型。符号结构不是旧时代遗物，它提供的是一致性、可审计性和组织协同。

Neuro-symbolic 的新语境

今天重新谈 neuro-symbolic，不是回到手写规则替代模型，而是让神经模型在开放环境中抽象出可编辑、可检查、可复用的符号结构。模型负责理解模糊现实，符号结构负责稳定流程与共享语义。

随后，概念漂移的讨论把问题从模型带回组织。AGI、Agent、Knowledge、AI-native 这些词会被人和模型反复使用，逐渐失去精确定义。更危险的是，模型会把流行词再生产给人，人再用这些词做决策，形成概念漂移循环。很多团队以为自己在讨论同一个概念，实际上每个人脑子里的对象都不同。

尚晏仪把这个问题说得很产品化：一家公司从零开始，最根本的是语义库。它认为“旅行”是什么，“订机票”是什么，“用户需求”是什么，这些定义会慢慢长出产品、流程和组织语言。如果这些定义不同步，组织协同就会变难；如果 AI 工具不断吸收和生成未经校准的内部文本，长期甚至会破坏组织的理智性。

热词会偷走判断力

“AGI” “Agent” “AI-native” “knowledge” 这些词如果不被重新定义，就会从思考工具变成思考替代品。团队需要先问：我们在这个场景里到底指什么？成功标准是什么？它对应哪些动作、数据、流程和边界？

谢天宝补充说，symbolic 世界的好处就是定义清楚、一致性强。如果能够连接 symbolic 和 neural，很多一致性问题可以先在 symbolic 层解决，再压回 neural 或 subsymbolic 层。这句话与整场对谈的产品问题其实互相呼应：Agent 不是只要更强模型，也需要更清楚的行动边界、语义定义和组织记忆。

7.1 本章小结

知识不只是事实库，还包括流程、情景、偏好和组织语义。Agent 系统如果要长期进入真实组织，就必须处理 neuro-symbolic 的结合：模型负责理解和泛化，符号结构负责一致性和协同。概念漂移不是语言洁癖，而是产品和组织风险；语义库会成为 Agent 公司的一种基础设施。

8 Startup 护城河：速度之后留下什么

最后一段讨论 startup 护城河。它表面上是创业经验，实际仍然延续前面的“积累问题”。大家常说创业公司优势是快，但尚晏仪指出，快不是完整逻辑。快的价值在于更早积累某些东西，而那些东西才可能成为护城河。

她把 AI 产品的早期竞争放在模型能力拐点里理解。当基础模型突然突破某个临界点，小公司如果最快找到一个场景并推出产品，就能抢到真空期。这个真空期会带来声量、品牌和用户心智。AI 时代的传播成本结构也不同于传统互联网买量：如果一个体验真的突破用户心理防线，用户会自发传播。

但声量并不够。旅行行业的例子说明，产品体验之后会进入供应链竞争。一个 App 做得早、体验好，如果酒店库存不够、价格不稳、履约不强，用户最终还是会回到供应链更强的平台。长期竞争会变成综合战：产品、品牌、供应链、用户行为、组织效率和再投入能力共同形成正循环。

速度不是护城河，速度带来的资产才是

快本身不能防御后来者。快只有在转化为品牌、用户习惯、供应链、数据、社区、artifact 或组织能力时，才可能形成长期壁垒。否则，基础模型能力一扩散，功能领先就会被迅速抹平。

谢天宝用 Google 搜索做类比：如果整个互联网在技术上都可以被 index，为什么后来者难以追

上 Google? 可能因为先做带来了更完整的 index、更丰富的 query 行为、更快的产品迭代和更强的品牌习惯。这个类比提醒我们, 先发优势不是玄学, 而是一组具体 artifact 的积累。

他们也讨论了什么样的积累更不容易被大厂吃掉。单纯数据可能被更多算力和更多用户追平; 但场景 artifact、AI-native 环境、供应链关系、社区信任、组织语义和产品细节更难复制。谷雨提到 NeoCognition 想升级数字环境本身, 让环境变得更 AI-native; 如果产出就是升级后的环境 artifact, 那么早做的积累就是产品本身, 而不是间接训练数据。

AI startup 的几类可积累资产

资产类型	为什么可能形成壁垒
品牌与心智	用户在新需求出现时先想到你, 降低后续获客成本。
供应链与履约	在交易型场景中, 体验最终要被库存、价格、服务和责任边界兑现。
场景 artifact	如果产出本身就是可复用环境、workflow 或知识结构, 早做会沉淀直接资产。
社区与信任	小公司能靠更近的用户关系形成粘性, 这种连接不易被纯功能替代。
组织语义库	对场景概念、流程和用户需求的长期校准, 会提高迭代效率。

结尾处尚晏仪强调人与人的连接。一个产品如果只是工具, 用户习惯可能迁移很快; 但如果产品意味着用户进入某个社区、确认某种身份、和一群人建立关系, 粘性会更强。这是小公司相对于大厂的重要机会: 不是在所有资源上取胜, 而是在具体人群里建立更密的信任网络。

8.1 本章小结

护城河不是一个静态名词, 而是动态竞争中的复利结构。AI 初创公司早期的快, 必须转化为能积累的资产; 中后期的胜负, 则取决于产品、供应链、社区、artifact 和组织效率是否能形成正循环。越容易被基础模型平台吸收的能力, 越不适合作为长期壁垒; 越贴近场景和用户关系的资产, 越可能留下。

9 总结: GUI Agent 的下一站不是更会点屏幕

整场对谈最终可以收束成一个判断: GUI Agent 的下一站不是更会点屏幕, 而是更会进入真实数字生活。进入真实世界之后, 问题会立刻变厚: 你如何评价它, 如何收费, 如何降低用户负担, 如何使用上下文, 如何持续学习, 如何维护组织语义, 如何形成可积累资产。这些问题没有一个能靠单次 demo 解决。

如果把三位嘉宾的观点压缩成一个结构, 可以得到下面这张表:

问题意识	对谈中的核心判断
GUI Agent 的本质	它不是“看屏幕并点击”的工具, 而是把用户目标转成数字环境行动的代理系统。
Operator 的启示	它证明方向被认可, 但也暴露“可见操作过程”不等于好产品体验。

问题意识	对谈中的核心判断
Benchmark 的边界	Benchmark 是 signal，不是 final answer；真实产品需要用户反馈和商业结果校准。
Reward 的困难	最终结果往往太远，主观反馈又太噪，Agent 产品需要持续试验和失败归因。
商业模式	Agent 更像智力劳动和交易代理，订阅与抽佣可能比广告更自然。
UDA 与动作空间	Deep Research、Coding、Browser、GUI 不应被切碎，成熟 Agent 应围绕目标选择动作。
长任务体验	能跑很久不是价值本身，减少等待、检查和上下文切换才是体验改进方向。
生成式 UI	有价值的是稳定组件加动态编排，而不是每次重新发明界面。
持续学习	上线后的模型不应只是推理工具，真实交互应成为能力和记忆的来源。
Memory	关键是表示、更新、使用三件事，不是把历史塞进 context。
概念漂移	热词会污染组织判断，公司需要语义库来维持共同定义。
Startup 护城河	快不是护城河，快带来的品牌、供应链、社区、artifact 和组织能力才是。

这期节目最好的地方，是它没有停在“Agent 很快会改变一切”的粗糙乐观里。相反，它反复展示了一个现实：当 Agent 真要进入用户生活和公司流程时，技术能力只是第一层。后面还有评价机制、商业分配、交互负担、隐私边界、持续学习、语义一致性和组织复利。GUI Agent 的未来，可能不属于最会演示鼠标点击的系统，而属于最能把这些厚问题逐步组织起来的系统。

附录：关键判断索引

关键判断	所对应的问题意识
<i>Deep Research</i> 、 <i>Coding</i> 、 <i>GUI Agent</i> 不能完全单独做，因为它们都发生在同一台电脑和同一个目标空间里。	这是 Unified Action Space 的直觉起点。用户目标天然跨工具，产品边界不应强迫用户搬运上下文。
<i>Operator</i> 让方向被认可，但它未必是好产品。	这区分了技术证明和用户体验。能展示 AI 操作，不等于用户愿意看 AI 慢慢操作。
<i>Benchmark</i> 分数越来越像 <i>signal</i> ，而不是终点。	它说明 evaluation 正从纯学术比较转向真实场景校准。分数仍有用，但解释边界必须更清楚。
旅行产品最终 <i>reward</i> 是卖出一张票，但这个 <i>reward</i> 离训练模型太远。	这揭示产业 Agent 的 <i>reward</i> 难题：最终结果明确，过程归因模糊，用户反馈主观且噪声很大。
<i>Agent</i> 输出的是智力劳动，不只是信息分发。	这是商业模式讨论的核心。广告卖注意力，Agent 更接近秘书、顾问、中介和代理服务。

关键判断	所对应的问题意识
能做一小时任务不是好事本身，如果同样质量 30 秒能完成，没人想等一小时。	这批评了长任务营销。自主执行能力必须转化为更低用户负担，而不是更长等待。
<i>Memory</i> 至少要问表示、更新、使用三件事。	这把长期记忆从“聊天记录”提升为系统设计问题。记忆必须可编辑、可遗忘、可回用。
热词会被模型和人反复传播，最后造成概念漂移。	这把 AI 时代的语言问题变成组织问题。没有语义库，公司会在流行词里失去共同判断。
快不是护城河，快之后积累了什么才是。	这是 startup 段落的核心。速度只有转化为品牌、供应链、社区、artifact 或组织能力时才有长期价值。